Exercises week 10

Last update 2026-10-26

These exercises aim to give you insight into and practical experience with

- Java functional programming, including immutable data, functional interfaces and lambda expressions;
- Java parallel streams and parallel array operations through mostly-functional programming.

Do this first

The exercises build on the lecture; Sestoft: *Java Precisely* chapter 24; and the lecture note section *9.4 Parallel k-means clustering and cache coherence*.

All the relevant Java code can be found on the Github page for week 10: (https://github.itu.dk/PCPP/PCPP2026-Public/tree/main/week10). The code for the exercises is in:

`.../week10/code-exercises/week10exercises/....`

Exercise 10.1 This exercise is about processing English words, using streams of strings. There is a list of some hundred thousand English words and names in file words.txt, which you can get from the course homepage or from https://raspi.itu.dk/people/sestoft/itu/PCPP/words.zip.

The exercises below should be solved without any explicit loops when possible. Most solutions are 1–5 lines long.

1. The TestWordStream.java file declares a method `readWords`. Check that you can use that method to read the `words.txt` and return its words as a Stream<String>. Write a Java stream expression to count the number words, and print that; it should be 104,334.
2. Write a Java stream expression to print only the first 20 words from the file. Use the `forEach` stream method with the method reference `System.out::println` as argument.
3. Write a Java stream expression to find and print all words that have at least 22 letters, using `filter`.
4. Write a Java stream expression that transforms all words to lowercase, using `map` and String's `toLowerCase` method, and then prints the first 20 transformed words.
5. Write a Java stream expression that transforms all words to lowercase and uses `filter` to throw away words that have only 1 letter or that contain characters other than `'a'` to `'z'`, then counts and prints the number of such words. The result should be 72,097 words.
6. Write a Java stream expression that does the transformation and filtering above, additionally strips away duplicates using method `distinct`, and then creates a List<String> of the resulting words, using the `collect` method and the collector `Collectors.toList()`, and assigns the result to a variable called `cleanWords`.

   In the questions below, turn `cleanWords` into a stream and use that instead of reading from file.
7. Write a method `boolean isPalindrome(String s)` that tests whether a word `s` is a palindrome: a word that is the same spelled forwards and backwards. Here you may use a for-loop. Using `cleanWords`, write a Java stream expression that finds all its palindromes and prints them. Among them should be "deified".
8. Using `cleanWords`, write a Java stream expression to group the words by length, using the `collect` method and a `Collectors.groupingBy` collector. That is, put all 2-letter words in one group, all 3-letter words in another group, and so on, and count the groups. The result will be map from length to a list of words, type Map<Integer,List<String>>, and should be bound to a variable called `lengthGroups`.
9. In continuation of the above, print the group of words of length 19. The word "chlorofluorocarbons" should be among them.
10. The file TestWordStream.java contains a method `counts` that turns a word into a Map<Character,Long> which shows how many times each letter appears in the word. For instance, for the word "persistent" the

resulting map will be `{e=2, i=1, n=1, p=1, r=1, s=2, t=2}`, saying that `'e'` appears 2 times in "persistent", `'i'` appears 1 time, and so on.

Now write a Java stream expression that transforms each word into its map of letter counts and prints it for the first 20 words.

11. If two words `s1` and `s2` have the same letter counts, as by `counts(s1).equals(counts(s2))`, then they are anagrams: they use the same letters the same number of times. For instance, "persistent" and "prettiness" are anagrams; both have letter counts `{e=2, i=1, n=1, p=1, r=1, s=2, t=2}`. Use the `collect` method on the word stream, a `groupingBy` collector and the `counts` method to find and print all groups of anagrams in `cleanWords`. Bind the result to a variable `anagram1` of type Map<Map<Character,Long>, List<String>>.

    Print all the groups of non-trivial anagrams: those where there is more than one word in the groups.

    Hint 1: One can turn the map `anagram1` into a set of its key-value pairs with `anagram1.entrySet()`.

    Hint 2: This can further be turned into a stream with `anagram1.entrySet().stream()`. Each element `kv` of this stream has a key `kv.getKey()` and a value `kv.getValue()`. Only the latter is interesting here; it is the list of words in the anagram group.

    For the `cleanWords` list the result should be 4712 non-trivial groups of anagrams, including `{a=1, c=1, e=1, r=1}=[acre, care, race]`.

12. Try to make a parallel version of the anagram-generating stream expression. Is it faster or slower than the sequential one?

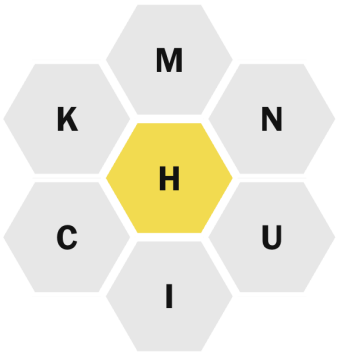
    Challenging

13. The previous subquestion probably shows that just slapping `.parallel()` on a stream pipeline may not make it faster. In this case, the reason likely is that `groupingBy` uses HashMaps internally for grouping; when applied to a parallel stream it repeatedly combines two HashMaps into a new HashMap, which is inefficient.

    Your task: Rewrite the previous solution to use the `groupingByConcurrent` collector, which internally uses a single thread-safe scalable ConcurrentHashMap and avoids the repeated combination of maps.

14. Define a new version `countsS` of the given `counts` method, using a stream expression and no loops.

Exercise 10.2 This exercise is about the New York Times Spelling Bee puzzles, published daily. Relevant code is in file TestSpellingBeeSolution.java. A spelling bee puzzle consists of 7 distinct letters, of which one letter is mandatory; for instance (`h`, `chikmnu`). The puzzle is usually shown as a honeycomb with the mandatory letter in the center, surrounded by the non-mandatory ones (order does not matter) like this:



A puzzle can be represented by this class:

```
class Puzzle {
  final char m;
  final Set<Character> W;

  public Puzzle(char m, Set<Character> W) {
    this.m = m;
    this.W = W;
  }

  public String toString() { ... }
}
```

A *solution* is an English word, at least four letters long, written using only the puzzle's seven letters (repeated use allowed) and where the mandatory letter appears. Some possible solutions to the given puzzle (`h`, `chikmnu`) are `chuck`, `hunch` and `munchkin`. A solution can be represented by a String.

The goal for a puzzle solver is to find as many solutions as possible. Of course, this is a simple computational task, provided one has a set of English words.

1. First, locate and download a set of words as mentioned in exercise 10.1. You need to read the file, clean the set, turn all words into lowercase, remove duplicates using the `distinct` method, and remove every word that is shorter than 4 letters or that uses letters other than `'a'` to `'z'`.

   Your task: Write a Java stream expression to do this file reading, transformation, discarding of duplicates and filtering, and turn the stream into a Set<String> called `cleanWords`. Print the number of elements in `cleanWords`. There should be 72,097 of them.

2. In slightly more formal terms, a Spelling Bee puzzle (m, W) consists of a mandatory letter m and a set W of precisely 7 distinct letters, such that $m \in W$, and where all letters are in the range a–z.

   A *solution* to puzzle (m, W) is a word from the above-mentioned list that contains letter m, that can be written using only letters from W, possibly with repetitions, and is at least 4 letters long.

   Your task: Add a method `boolean hasSolution(String s)` to class Puzzle. It should return true if `s` is a solution to the puzzle, false otherwise. Use the `letterSet` method and stream expressions, no loops.

3. Add a method `Stream<String> solutions(List<String> cleanWords)` to class Puzzle. It should produce a stream of the words from `cleanWords` that are solutions to the puzzle. Use stream expressions, no loops.

4. Compute and print the stream of solutions to `new Puzzle('h', letterSet("chikmnu"))`, using words from `cleanWords`. There should be 22 such solutions.
5. Create a stream of all the puzzles that can be made from words in `cleanWords`. You can do this by transforming each word into its letter set `W`, filtering away those lettersets that do not contain precisely 7 letters, removing duplicate letter sets, and then for each letter `m` from `W`, create a puzzle out of (`m`, `W`). You will need `map`, `filter`, `distinct` and `flatMap`. Finally, turn this stream into a list `puzzles` of type `List<Puzzle>`. Print the number of possible puzzles. There should be 62,174 of them.
6. Print the first 20 puzzles.

Challenging

7. A puzzle is a good one if it has at least 10 solutions and at most 80. (If there are 250 solutions, nobody has the patience to solve the puzzle).

   Using `puzzles` and Java stream expressions, for each puzzle count its solutions and keep it only if it has 10–80 solutions. Turn the resulting stream into a list `goodPuzzles` and print the number of them. There should be 28,787 of them.

   Warning: This computation is quite slow; it takes 8 minutes on a modern fast laptop computer.
8. Parallelise the stream computation in the previous question. How much faster is the parallel version on your computer?

Challenging

Exercise 10.3 In this exercise we will use parallel array operations to experimentally investigate the assertion that the count $\pi(n)$ of prime numbers less than or equal to n is proportional to $n/\ln(n)$, where $\ln(n)$ is the natural logarithm of n. More precisely, the ratio $\pi(n)/(n/\ln(n))$ converges to 1 for large n. This is known as the prime number theorem.

1. Create an `int` array `a` of size N, for instance for $N = 10{,}000{,}001$. Use method `parallelSetAll` from utility class Arrays to initialize position `a[i]` to 1 if `i` is a prime number and to 0 otherwise. You may use method `isPrime` from e.g. file TestCountPrimesThreads.java.
2. Use method `parallelPrefix` from utility class Arrays to compute the prefix sums of array `a`. After that operation, the new value of `a[`i`]` should be the sum of the old values `a[0..`i`]`. Therefore, the new value of `a[`i`]` is the count of prime numbers smaller than or equal to `i`, that is, $\pi(i)$. For instance, the value of `a[10_000_000]` should be 664,579.
3. Use a for-loop to print the ratio between `a[`i`]` and $i/\ln(i)$ for 10 values of `i` equally spaced between $N/10$ and N. Does the ratio converge towards 1? Does it converge fast?